

Lab 3

```
In [1]: import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
```

a)

```
In [2]: train_df = pd.read_csv("train.tsv", names = ['Stars', 'Docid', 'Text'], sep="\t", header=0)
```

```
In [3]: dev_df = pd.read_csv("dev.tsv", names = ['Stars', 'Docid', 'Text'], sep="\t", header=0)
```

```
In [4]: test_df = pd.read_csv("test.tsv", names = ['Stars', 'Docid', 'Text'], sep="\t", header=0)
```

```
In [5]: train_df.head()
```

```
Out[5]:
```

	Stars	Docid	Text
0	4	z8DDztUxuloHYHddDL9zQ	So let me set the scene first, My church socia...
1	2	BleDBg4MrEd1NwWRIFHLQQ	Decent but terribly inconsistent food. I've ha...
2	4	NJHPiW30SKhltD5E2jqpHw	Looks aren't everything..... This little di...
3	2	nnS89FMpIH7NPjYHmug	Being a creature of habit anytime I want good ...
4	2	FYxSugh9PGrX1PR0BHBlw	I recently told a friend that I cant figure ou...

b)

```
In [6]: mapping = {4:1, 2:0}
```

```
In [7]: train_df['Stars']
```

```
Out[7]: 0      4
        1      2
        2      4
        3      2
        4      2
        ..
        1995    4
        1996    2
        1997    4
        1998    4
        1999    2
        Name: Stars, Length: 2000, dtype: int64
```

```
In [8]: y_train = np.array([mapping.get(v) for v in train_df['Stars']])
```

```
In [9]: y_dev = np.array([mapping.get(v) for v in dev_df['Stars']])
```

```
In [10]: y_train
```

```
Out[10]: array([1, 0, 1, ..., 1, 1, 0])
```

```
In [ ]:
```

```
In [11]: bagOfWords = CountVectorizer(lowercase=True, stop_words="english", min_df=2,
```

```
In [12]: X_train = bagOfWords.fit_transform(train_df['Text'])
```

```
In [13]: feature_names = bagOfWords.get_feature_names_out()
         print(len(feature_names))
```

```
5579
```

```
In [ ]:
```

```
In [14]: X_dev = bagOfWords.transform(dev_df['Text'])
```

```
In [15]: first_doc_vector = X_dev[0]
         nonzero_indices = first_doc_vector.nonzero()[1]
```

```
In [16]: for idx in nonzero_indices:
         print(f"{feature_names[idx]}: {first_doc_vector[0, idx]}")
```

average: 1
business: 1
chef: 1
chicken: 1
china: 2
deserves: 1
does: 1
enjoyed: 1
fan: 1
food: 1
forward: 1
friendly: 1
general: 1
good: 2
grand: 1
great: 1
items: 1
just: 1
lady: 1
look: 1
management: 1
menu: 3
new: 1
online: 1
opened: 1
original: 2
pepper: 1
phone: 1
place: 1
portion: 1
price: 1
really: 1
recent: 1
restaurant: 1
return: 1
site: 1
size: 1
steak: 1
tasty: 1
told: 1
trying: 1
tso: 1
wanted: 1
years: 1

```
In [17]: NB = MultinomialNB(alpha=1.0)
```

```
In [18]: NB.fit(X_train, y_train)
```

```
Out[18]: ▾ MultinomialNB
MultinomialNB()
```

```
In [19]: y_dev_pred = NB.predict(X_dev)
```

```
In [20]: rev_mapping = {1:4, 0:2}
```

```
In [21]: predictions = np.array([rev_mapping.get(v) for v in y_dev_pred])
```

```
In [22]: print("docid\t\t\tprediction")
for i in range(10):
    print(f"{dev_df.iloc[i]['Docid']}\t\t{predictions[i]}\n")
```

docid	prediction
ZSJnW6faaNFQoqq4ALqYg	4
Rcbv11hm5AYEwZyqYwAvg	4
rkRTjhu5szaBggeFVcVJlA	4
dhmeDsQGUS1FXMLs49SWjQ	4
z9zfIMYmRRCE4ggf0IieEw	4
Xtb3pGSh39bqcozkBECw	2
DOUfLAGzxLsXG6x0mR1w	2
ØRxCEWUReØ8CTcZt95F4AQ	2
MzUg5twEcCydØX6lBMP2Lg	2
uNlw2D5CYKkØwjNxLtYw	4

My approach was to use a Naive Bayes model to make the predictions. I first use a bag of words vectorizer to get the vectors of the documents. I set the parameters to make it all lowercase, remove stop words, make it so that a word must appear in at least 2 documents to be kept, and a word must appear in no more than 90% of all documents. I made a generic Naive Bayes model with default parameters and predicted on the data.

c)

```
In [23]: def evaluate(true_values, pred_values):
```

```

tp=0
fp=0
fn=0
for i in range(len(true_values)):
    if true_values[i] ==1 and pred_values[i] ==1:
        tp+=1
    elif true_values[i]==0 and pred_values[i] ==1:
        fp+=1
    elif true_values[i]==1 and pred_values[i] ==0:
        fn+=1
precision = 0
if (tp + fp) > 0:
    precision = tp/(tp + fp)
recall = 0
if (tp + fn) > 0:
    recall = tp/(tp + fn)
f1_score = 0
if (precision + recall) > 0:
    f1_score = 2*(precision*recall)/(precision+recall)

return precision, recall, f1_score

```

```
In [24]: precision, recall, f1_score = evaluate(y_dev,y_dev_pred)
```

```
In [25]: print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1-score: {f1_score}")
```

Precision: 0.8254764292878636

Recall: 0.823

F1-score: 0.8242363545317977

```
In [26]: devCopy = dev_df.copy()
```

```
In [27]: devCopy['ActualVal'] = y_dev
```

```
In [28]: devCopy['PredVal'] = y_dev_pred
```

```
In [29]: incorrect = devCopy[devCopy['ActualVal']!=devCopy['PredVal']]
```

```
In [30]: selected_rows = incorrect[['ActualVal','PredVal','Text']].head(3)
i = 1
for index, row in selected_rows.iterrows():
    act = row['ActualVal']
    pred = row['PredVal']
    text = row['Text']
    print(f"Example {i}, ACTUAL {act}, PRED {pred}: \n{text}\n")
    i += 1
```

Example 1, ACTUAL 0, PRED 1:

Olive Garden used to be a favorite of the family, recently they cut back the menu extensively and many of our favorites are gone. I suggest checking the menu online before coming to see what's left.

Example 2, ACTUAL 1, PRED 0:

Our Las Vegas friend suggested duck tacos after hanging out and having some drinks for a late night/ early morning (around 4am) snack. All we needed to hear was tacos and we were in. The small bar area of this place is open 24/7 so we stopped in for some half priced tapas (happy hour from midnight to 8 am) and to chat with Dave, our friend's friend and an incredibly cool guy that was working the overnight shift. It was 4am and I didn't want to be stuffed before going to bed, so I only ordered a few dishes. I had the steak skewers with teriyaki glaze and the garlic cheese bread. Both were good, but the garlic cheese bread was really good in its garlicky and cheesy-ness. I also had one of the duck tacos from one of the others' plates and it was decent, but if I got it again I'd get it without the creamy tomato cilantro sauce. Overall, I'd say the food is worth 3 or 3.5 stars, but Dave's coolness bumps this location up to a solid 4 stars.

Example 3, ACTUAL 0, PRED 1:

Stopped in here for lunch the other day. Quiet little independent Chinese restaurant that also has a Hawaiian style menu. Not many options for vegetarians, no tofu dishes to be found. Didn't really dig my noodles + veggies + pineapple dish, hence the 2 star rating.

d)

```
In [31]: SVM = LinearSVC(C=1.0, class_weight='balanced')
```

```
In [32]: SVM.fit(X_train,y_train)
```

```
/Users/sohumpohane/.pyenv/versions/3.10.14/lib/python3.10/site-packages/sklearn/svm/_classes.py:32: FutureWarning: The default value of `dual` will change from `True` to `auto` in 1.5. Set the value of `dual` explicitly to suppress the warning.
  warnings.warn(
```

```
Out[32]: LinearSVC
LinearSVC(class_weight='balanced')
```

```
In [33]: y_dev_pred = SVM.predict(X_dev)
```

```
In [ ]:
```

```
In [34]: predictions = np.array([rev_mapping.get(v) for v in y_dev_pred])
```

```
In [35]: print("docid\t\t\tprediction")
         for i in range(10):
             print(f"{dev_df.iloc[i]['Docid']}\t\t{predictions[i]}\n")
```

docid	prediction
ZSJnW6faaNFQoqq4ALqYg	4
Rcbv11hm5AYEwZyqYwAvg	2
rkRTjhu5szaBggeFVcVJlA	4
dhmeDsQGUS1FXMLs49SWjQ	4
z9zfIMYmRRCE4ggf0IieEw	4
Xtb3pGSh39bqcozkBECw	2
DOUflAGzxLsXG6x0mR1w	2
0RxCEWURe08CTcZt95F4AQ	2
MzUg5twEcCyd0X6lBMP2Lg	2
uNlw2D5CYKk0wjNxLtYw	2

```
In [36]: precision, recall, f1_score = evaluate(y_dev,y_dev_pred)
```

```
In [37]: print(f"Precision: {precision}")
         print(f"Recall: {recall}")
         print(f"F1-score: {f1_score}")
```

```
Precision: 0.7989949748743719
Recall: 0.795
F1-score: 0.7969924812030076
```

```
In [ ]:
```

```
In [38]: devCopy = dev_df.copy()
```

```
In [39]: devCopy['ActualVal'] = y_dev
```

```
In [40]: devCopy['PredVal'] = y_dev_pred
```

```
In [41]: incorrect = devCopy[devCopy['ActualVal']!=devCopy['PredVal']]
```

```
In [42]: selected_rows = incorrect[['ActualVal', 'PredVal', 'Text']].head(3)
i = 1
for index, row in selected_rows.iterrows():
    act = row['ActualVal']
    pred = row['PredVal']
    text = row['Text']
    print(f"Example {i}, ACTUAL {act}, PRED {pred}: \n{text}\n")
    i += 1
```

Example 1, ACTUAL 1, PRED 0:

Meeting a friend for lunch, we had a little miscommunication. He went to the wrong one, so I got to see two CFA's for lunch in one day. Like the other, this one was very busy at lunch time. I guess people are thinking that chicken is healthier than the other options in this shopping plaza? Large seating area, they have a couple of staff wandering around to improve things. The drink lady is a nice touch and she's super friendly. She even refilled the drink I'd brought from the other CFA, though that's probably not the usual deal. My friend and I took up the table for a good hour and no one troubled us at all!

Example 2, ACTUAL 0, PRED 1:

Olive Garden used to be a favorite of the family, recently they cut back the menu extensively and many of our favorites are gone. I suggest checking the menu online before coming to see what's left.

Example 3, ACTUAL 1, PRED 0:

Our Las Vegas friend suggested duck tacos after hanging out and having some drinks for a late night/ early morning (around 4am) snack. All we needed to hear was tacos and we were in. The small bar area of this place is open 24/7 so we stopped in for some half priced tapas (happy hour from midnight to 8 am) and to chat with Dave, our friend's friend and an incredibly cool guy that was working the overnight shift. It was 4am and I didn't want to be stuffed before going to bed, so I only ordered a few dishes. I had the steak skewers with teriyaki glaze and the garlic cheese bread. Both were good, but the garlic cheese bread was really good in its garlicky and cheesy-ness. I also had one of the duck tacos from one of the others' plates and it was decent, but if I got it again I'd get it without the creamy tomato cilantro sauce. Overall, I'd say the food is worth 3 or 3.5 stars, but Dave's coolness bumps this location up to a solid 4 stars.

My approach was to use a SVM model to make the predictions. I used the bag of words vectorizer to get the vectors of the documents. I made a generic SVM model with default parameters and predicted on the data.

e)

I try a Tfidf vectorizer for better result and use unigram and bigrams given by the

parameter ngram_range, also the words are all lowercase

```
In [43]: vectorizer = TfidfVectorizer(lowercase=True,min_df=2,ngram_range=(1, 2))
```

```
In [44]: X_train_new = vectorizer.fit_transform(train_df['Text'])
```

```
In [ ]:
```

```
In [45]: NB2 = MultinomialNB(alpha=1.0)
```

```
In [46]: NB2.fit(X_train_new, y_train)
```

```
Out[46]: ▼ MultinomialNB  
MultinomialNB()
```

```
In [47]: X_dev_new=vectorizer.transform(dev_df['Text'])
```

```
In [48]: y_dev_pred = NB2.predict(X_dev_new)
```

```
In [49]: precision, recall, f1_score = evaluate(y_dev,y_dev_pred)
```

```
In [50]: print(f"Precision: {precision}")  
print(f"Recall: {recall}")  
print(f"F1-score: {f1_score}")
```

Precision: 0.8799559471365639

Recall: 0.799

F1-score: 0.8375262054507338

```
In [51]: def save_predictions(data_df, predictions, filename):  
    with open(filename, 'w') as f:  
        for i in range(len(predictions)):  
            f.write(f"{data_df.iloc[i]['Docid']}\t{predictions[i]}\n")
```

```
In [52]: save_predictions(dev_df,y_dev_pred,'spohane1.txt')
```

```
In [ ]:
```